

Online gaming systems require efficient sorting algorithms to manage player rankings, leaderboards, and match-making. This paper presents a comprehensive analysis of sorting algorithms in gaming contexts, covering algorithm selection, performance analysis, online gaming, factorial experiment, merge sort, quick sort, data distribution, and conclusions.

Introduction

Online gaming systems have experienced unprecedented growth, with millions of concurrent players generating massive amounts of data. The choice of sorting algorithm significantly impacts system performance, particularly when dealing with gaming-specific data distributions.

This paper addresses the critical need for empirical performance analysis of sorting algorithms in gaming contexts.

Research Objectives

Our study aims to:

Evaluate the performance of Merge Sort and Quick Sort on gaming-relevant data distributions

Analyze the impact of data size and distribution patterns on algorithm performance

Provide empirical guidelines for algorithm selection in online gaming systems

Quantify the trade-offs between execution time, memory usage, and throughput

Contributions

The main contributions of this work include:

A comprehensive factorial experiment design (2³) for sorting algorithm evaluation

Implementation of realistic gaming data generators using inverse transform sampling

Detailed statistical analysis including ANOVA, confidence intervals, and effect size calculations

Practical recommendations for algorithm deployment in production gaming systems

Related Work

Sorting algorithms have been extensively studied in computer science literature [7]. However, most studies focus on theoretical complexity analysis.

Recent work by [5] examined database query optimization for gaming systems but did not address in-memory sorting algorithms.

Our work differs by focusing specifically on the intersection of sorting algorithm performance and gaming data characteristics.

Methodology

Experimental Design

We employed a full factorial experiment design (2³) to systematically evaluate the performance of sorting algorithms across different data sizes and distributions.

Factor A - Data Size: 10,000 vs 100,000 elements

Factor B - Data Distribution: Exponential vs Quasi-sorted

Factor C - Algorithm: Merge Sort vs Quick Sort

This design resulted in 8 unique combinations, with each experiment repeated 10 times to ensure statistical reliability.

Data Generation

Exponential Distribution Gaming scores typically follow exponential distributions, where most players achieve moderate scores.

where $U \sim Uniform(0, 1)$ and $\lambda = 0.001$ to achieve a mean score of approximately 1,000 points.

Quasi-sorted Distribution Leaderboards in gaming systems are frequently updated, creating datasets that are nearly sorted.

Algorithm Implementation

Merge Sort We implemented a standard divide-and-conquer Merge Sort with $O(n \log n)$ guaranteed time complexity.

Quick Sort Our Quick Sort implementation uses the median-of-three pivot selection strategy to improve performance on non-uniform data.

Performance Metrics

For each experiment, we measured:

Execution Time: Wall-clock time using high-precision timers

Memory Usage: Peak memory consumption during sorting

Throughput: Elements processed per second

Speedup: Relative performance between algorithms

Statistical Analysis

We conducted comprehensive statistical analysis including:

Descriptive statistics with confidence intervals (95%)

Analysis of Variance (ANOVA) to identify significant factors

Post-hoc analysis with Bonferroni correction

Effect size calculation using Cohen's d

Outlier detection using the IQR method

Results and Analysis

Distribution Analysis of Execution Times

Figure 1 presents a detailed analysis of execution time distributions through box plots, showing performance differences across data sizes and distributions.

[H] [width=0.45]plots/boxplots_distribuicao.pngBoxplot analysis showing distribution of execution times by algorithm and data characteristics.

Overall Performance Comparison

Figure 2 presents a direct comparison between Merge Sort and Quick Sort across different experimental configurations.

[H] [width=0.45]plots/comparativo_barras.pngPerformance comparison between Merge Sort and Quick Sort across different data sizes and distributions.

Scalability Analysis

Figure 3 demonstrates how each algorithm scales with increasing data size.

[H] [width=0.45]plots/escalabilidade_algoritmos.pngScalability analysis showing algorithm performance as data size increases.

Performance Metrics Correlation

Figure 4 presents the correlation matrix between different performance metrics.

[H] [width=0.42]plots/heatmap_correlacao.pngCorrelation heatmap between performance metrics (time, memory, throughput).

Factor Interaction Analysis

Figure 5 shows how different experimental factors interact with each other.

[H] [width=0.45]plots/interacao_fatores.pngInteraction analysis between experimental factors (algorithm, size, distribution).

Memory vs Time Trade-off

Figure 6 analyzes the trade-off between memory usage and execution time.

[H] [width=0.42]plots/memoria_vs_tempo.pngAnalysis of the relationship between memory usage and execution time.

Execution Time Distribution

Figure 7 shows the detailed distribution of execution times.

[H] [width=0.45]plots/distribuicao_tempos.pngDistribution of execution times for different experimental configurations.

Throughput Analysis

Figure 8 compares the throughput (elements processed per second) of the algorithms.

[H] [width=0.45]plots/throughput_comparacao.pngThroughput comparison between algorithms. Metrics of elements processed per second.